AI Alert Agent — Project Summary for Management

Project: AI-Powered Alert Investigation Agent **Team:** Platform / Infrastructure **Last Updated:** June 2026

What Is This System?

When a production alert fires (e.g. a pod is consuming too much memory, a server is down, Kafka is lagging), the on-call engineer currently has to manually log into Grafana, Kubernetes, and Loki to find the root cause — often taking 15–30 minutes per alert.

The **AI Alert Agent** automates this investigation. It receives the alert, queries all relevant systems in parallel using AI, and posts a structured Root Cause Analysis (RCA) directly to the team's Slack channel — in under 2 minutes.

What We Built

Core System

Component	What It Does
Webhook Server	Receives alerts from Alertmanager (the Prometheus alerting system)
AI Investigation Agent	Uses GPT-4o to reason over live data and produce an RCA
Kubernetes Tool (k8s-mcp)	Queries pod status, events, logs, deployment state
Prometheus Tool (prometheus-mcp)	Queries CPU, memory, disk, network, probe, and node metrics
Loki Tool (loki-mcp)	Queries application and infrastructure logs
Kafka Tool (kafka-mcp)	Queries consumer lag, broker throughput, topic health
Slack Reporter	Posts formatted RCA with color-coded severity to the right channel

All components run in a **single Docker container** — one deployment, no external dependencies beyond OpenAI API.

What We Delivered (This Phase)

1. Alert Coverage — 153 Rules Fully Mapped

We analyzed all 153 production alerting rules and ensured every alert type has:

- A one-line human-readable description (alert catalog)
- The correct investigation tools assigned

- Specific hints for the AI on what to look for

Alert types now covered:

Category	Count	Example Alerts
Kubernetes pod	15+	CPU/memory limits, OOM kill, eviction, anomalies, restarts
EC2 host infrastructure	21	Host down, out of memory, swap, disk latency, clock skew, temperature, RAID
Kafka / MSK	10+	Consumer lag, no messages, broker throughput
Blackbox probes	3	HTTP probe down, high latency, TCP probe failed
TLS certificates	1	Certificate expiring soon

---

2. Slack Channel Routing

Previously all alerts went to a single Slack webhook. We added a **routing configuration** (`routing.yaml`) that lets the team direct alerts to different Slack channels based on alert labels — without touching code or rebuilding the container.

Routing rules support:

- Exact label match (e.g. `severity: critical` AND `stage: prod`)
- Regex label match (e.g. all `EC2Host*` alerts → `#infra-alerts`)
- First-match-wins ordering (same as Alertmanager)
- A default fallback channel

To update routing: edit `routing.yaml` on the server and restart the container. No code change, no rebuild.

---

3. Rich Slack RCA Format

Each investigation now produces a two-part Slack message:

Header (color-coded by severity — red for critical, yellow for warning):

- Alert name + severity
- Namespace / pod / region / started time
- One-line summary from Prometheus annotations
- Direct link to the Prometheus graph that triggered the alert

Body:

- Findings (bullet points of what the AI observed)
- Probable Root Cause
- Recommended Actions (numbered list)
- Auto-truncated if the report exceeds Slack's display limit

---

4. New Diagnostic Tools

We added two new investigation tools to expand what the AI can query:

`get_node_advanced` **(Prometheus)** — fetches EC2 host signals in a single call: swap usage, inode free space, disk read/write latency, failed systemd services, conntrack fill %, clock offset, hardware temperature, and RAID status.

`get_broker_throughput` **(Kafka)** — fetches bytes-in and bytes-out per second for a specific Kafka topic to investigate high-throughput alerts.

---

5. Reliability Improvements

Improvement	Detail
Deterministic fallback	If OpenAI API is unavailable or quota exceeded, the agent falls back to a rule-based RCA using pre-fetched metrics — no silent failures
Alert deduplication	Same alert firing twice within 15 minutes is suppressed (configurable)
Slack retry with backoff	Failed Slack posts retry up to 3 times (2s → 4s → 8s delay)
k8s graceful degradation	If no Kubernetes credentials are available (local dev), k8s tools return a clear error instead of crashing the container
Body truncation	RCA body is capped at 3,800 characters with a notice — prevents Slack API rejections

---

6. Observability

The agent exposes Prometheus metrics at `/metrics`:

Metric	What It Measures
`alerts_received_total`	Total alerts received per alert name
`alerts_accepted_total`	Alerts that passed dedup and allowlist filters
`alerts_deduplicated_total`	Duplicate alerts suppressed
`llm_investigations_total`	LLM calls by outcome (success / fallback / error)
`slack_posts_total`	Slack posts by outcome (success / error)

---

7. Test Coverage

63 automated tests covering:

- Alert routing (match/regex/first-match/fallback)
- Slack formatting (header, body, truncation, color)

- Alert context classification (pod vs host vs Kafka vs probe)
- RCA formatting and deterministic fallback
- Webhook dedup and filter logic

Deployment

Environment	Status
Local (docker-compose)	✅ Running
Kubernetes (dozee-dev)	Manifests ready
Kubernetes (dozee-pro)	Manifests ready

Single container per cluster. No database, no external queue. Logs saved to a mounted volume (`/app/logs`).

Future Improvements

The following improvements are recommended for the next phase, prioritized by business impact:

High Priority

Improvement	Business Value
Persistent dedup store (Redis/SQLite)	Currently dedup resets on container restart — duplicate RCAs can fire during deployments
Runbook links in RCA	Each alert links directly to the runbook in the Slack message — reduces MTTR for common issues
`/webhook/test` endpoint	On-call engineers can test a sample alert and see the RCA without waiting for a real incident
Routing hot-reload	Edit `routing.yaml` without restarting the container — zero-downtime routing updates

Medium Priority

Improvement	Business Value
Multi-alert correlation	When 5 pods on the same node go down simultaneously, produce one grouped RCA instead of 5 separate ones
Historical comparison	Flag alerts that have fired 3+ times in the past week as "recurring" — helps prioritize permanent fixes over repeated mitigations
Confidence score	AI rates its certainty (Low / Medium / High) — low-confidence RCAs are flagged so engineers know to investigate manually

Improvement	Business Value
Structured JSON logs	Current logs use `print()` — structured logging enables log queries in Loki/CloudWatch for audit and debugging

Lower Priority / Nice to Have

Improvement	Business Value
Grafana dashboard	Visualize alert volume, LLM success rate, and Slack delivery reliability over time
Alert storm protection	Bounded queue prevents runaway thread creation during mass-firing incidents
Cost optimization	Route `info`-severity alerts to GPT-4o-mini instead of GPT-4o — same quality for low-stakes alerts at ~10x lower API cost
Helm chart	Package as a Helm chart for GitOps-native deployment and easier version management across clusters
Slack slash commands	`/rca replay <alert>` — replay a past alert through the agent; `/rca status` — show current agent health from Slack

Cost Estimate (OpenAI)

Approximate per-investigation cost using GPT-4o:

Alert type	Avg tokens (in + out)	Approx cost
Simple pod alert	~4,000	~$0.02
Complex EC2 host alert	~8,000	~$0.04
Kafka lag with logs	~10,000	~$0.05

At 50 investigations/day: **~$1–2.50/day** depending on alert mix. Cost optimization (GPT-4o-mini for info/warning severity) could reduce this by 60–70%.

Summary

Area	Before	After
Alert investigation time	15–30 min manual	< 2 min automated
Alert types covered	~7 (basic pods only)	153 rules across 5 categories
Slack routing	Single channel	Per-channel by label rules
RCA quality	Manual, inconsistent	Structured, reproducible, severity-colored
Reliability	No fallback	LLM fallback + retry + dedup

Area	Before	After
Observability	None	Prometheus metrics + `/metrics` endpoint
Test coverage	0	63 automated tests